Université / Établissement

Module : Sécurité Informatique
Sécurité des Applications Web

Compte-Rendu des Travaux Pratiques

Farida Zran
3ème Année Licence Génie Logiciel (SLAM)

Date : 16 Octobre 2026

Table des matières

1 Jour 1 : Configuration de l'environnement et architecture **2**

2 Jour 2 : Développement de la version vulnérable (V1) **3**
2.1 Développement du module d'authentification et Injection SQL 3
2.2 Espace de commentaires et XSS Stockée 4

3 Jour 3 : Analyse des risques et préparation de la remédiation **5**
3.1 Synthèse des vulnérabilités identifiées 5

4 Jour 4 : Développement de la Version 2 - Sécurisée **6**
4.1 Correction de l'Injection SQL par les requêtes préparées 6
4.2 Sécurisation et hachage robuste des mots de passe 6
4.3 Neutralisation des failles XSS 7

1 Jour 1 : Configuration de l'environnement et architecture

La première journée a été consacrée à la mise en place de l'environnement technique conteneurisé ainsi qu'à la définition globale du projet :

- **Mise en place de l'environnement Docker :** Configuration et lancement d'un environnement multi-conteneurs (via Docker Compose) isolant un serveur web Apache/PHP 8.2, un SGBD MySQL 8.0 et une interface d'administration phpMyAdmin.
- **Validation des services :** Vérification des accès locaux et des ports de communication (serveur web sur le port 8080 et phpMyAdmin sur le port 8081).
- **Choix architecturaux :** Validation du sujet de l'application (un mini-blog interactif regroupant authentification, gestion des sessions et espace de commentaires).

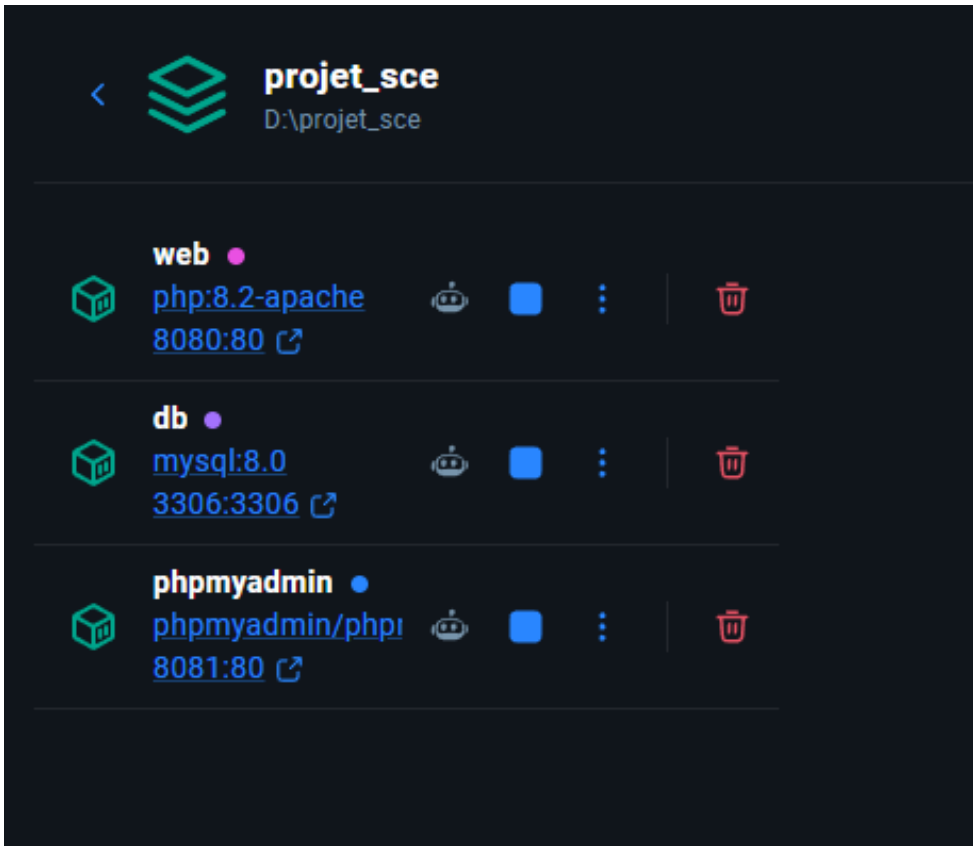


FIGURE 1 – Architecture Docker et conteneurs actifs

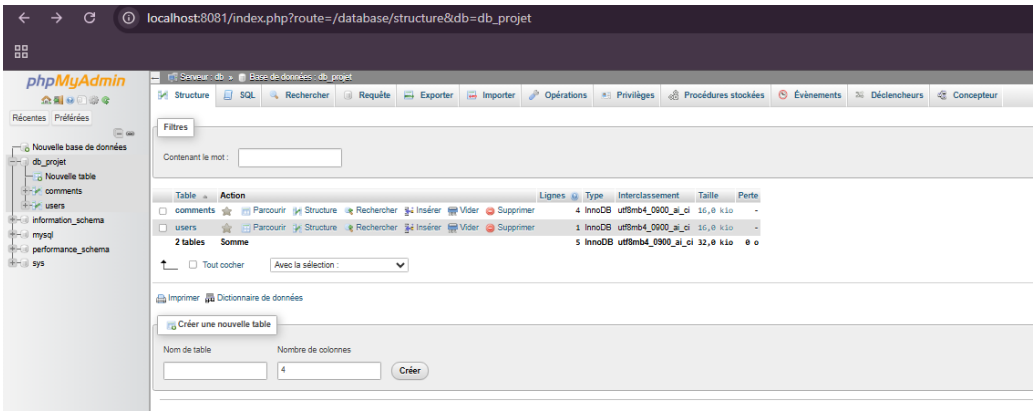


FIGURE 2 – Interface phpMyAdmin et structure de la base de données

2 Jour 2 : Développement de la version vulnérable (V1)

La deuxième journée a été consacrée au développement de l'application Mini-blog dans sa version non sécurisée (V1) et à la mise en évidence pratique de deux vulnérabilités majeures de l'OWASP Top 10.

2.1 Développement du module d'authentification et Injection SQL

La page `login.php` a été conçue en effectuant une concaténation directe des variables reçues via la méthode POST dans la requête SQL, sans utiliser de requêtes préparées.

Listing 1 – Implémentation vulnérable de l'authentification (V1)

```
$sql = "SELECT * FROM users WHERE username = '$user' AND password =
    '$pass'";
$result = $pdo->query($sql);
```

Démonstration de la faille (SQLi) : En saisissant le payload `admin' OR '1'='1` dans le champ du nom d'utilisateur, la condition logique est modifiée pour devenir toujours vraie, permettant de contourner l'authentification.

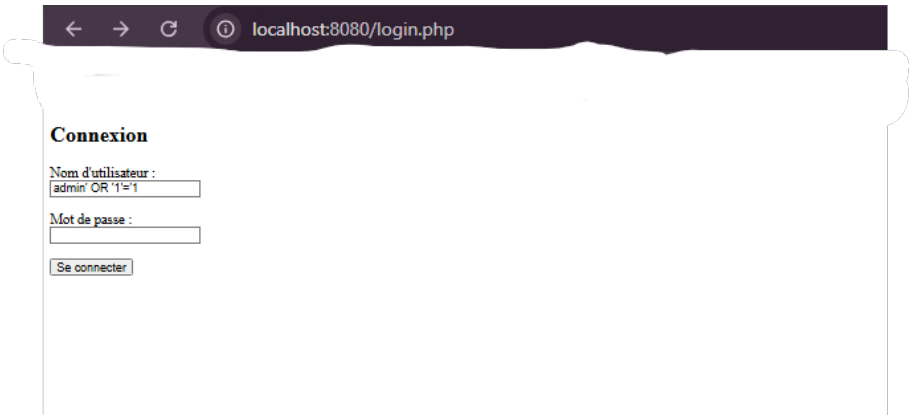


FIGURE 3 – Interface de connexion de la version vulnérable (V1)

2.2 Espace de commentaires et XSS Stockée

La page `index.php` permet de publier des commentaires. Le texte saisi est inséré et récupéré de la base de données sans aucun échappement des caractères spéciaux HTML.

Listing 2 – Insertion vulnérable d'un commentaire XSS

```
$insert = $pdo->prepare("INSERT INTO comments (user_id, content)
    VALUES (?, ?)");
$insert->execute([$user_id, $content]);
```

Démonstration : L'injection de la charge utile `<script>alert('XSS Stockee')</script>` s'exécute automatiquement dans le navigateur de tout utilisateur consultant le flux.

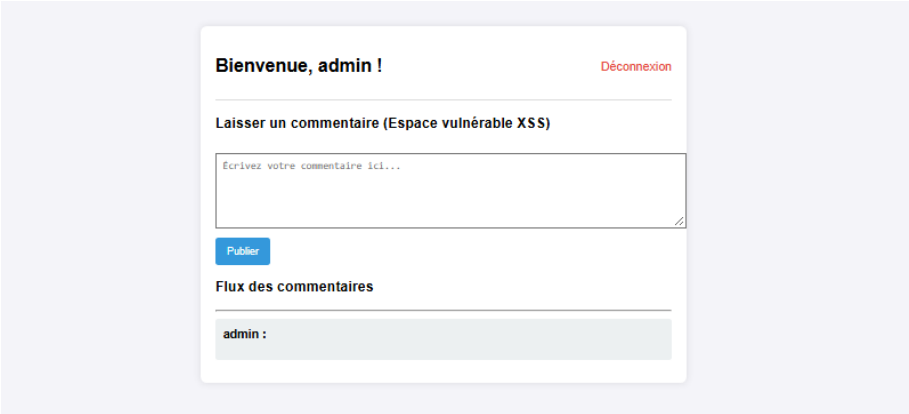


FIGURE 4 – Connexion réussie et espace de commentaires vulnérable

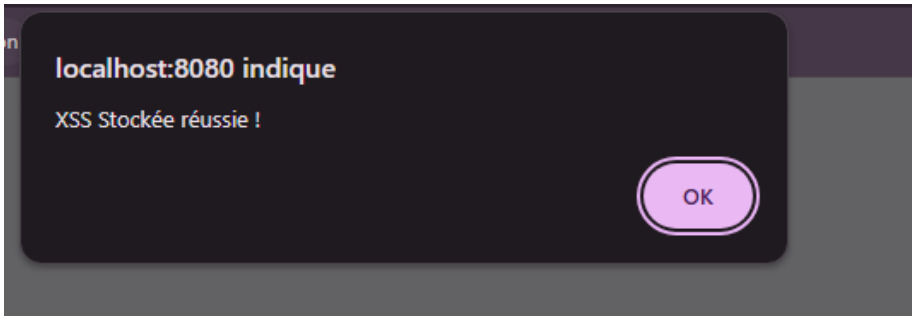


FIGURE 5 – Exécution de la pop-up XSS stockée dans le navigateur

3 Jour 3 : Analyse des risques et préparation de la remédiation

La troisième journée a permis de documenter les scénarios d'attaque constatés lors des tests d'intrusion sur la version V1 et de structurer la stratégie de correction pour la version V2.

3.1 Synthèse des vulnérabilités identifiées

— **Injection SQL :** Contournement total de l'authentification admin sans connaître le mot de passe.
— **XSS Stockée :** Vols potentiels de cookies de session et altération de l'affichage de la page.
— **Absence de hachage :** Stockage des mots de passe en clair en base de données, représentant un risque critique en cas de fuite (dump).

4 Jour 4 : Développement de la Version 2 - Sécurisée

Cette quatrième journée a été dédiée à l'application des bonnes pratiques de développement sécurisé (Secure Coding).

4.1 Correction de l'Injection SQL par les requêtes préparées

Les requêtes dynamiques par concaténation ont été remplacées par des requêtes préparées utilisant l'API PDO.

Listing 3 – Authentification sécurisée avec PDO

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = :
    username');
$stmt->execute(['username' => $username]);
$user = $stmt->fetch();
```

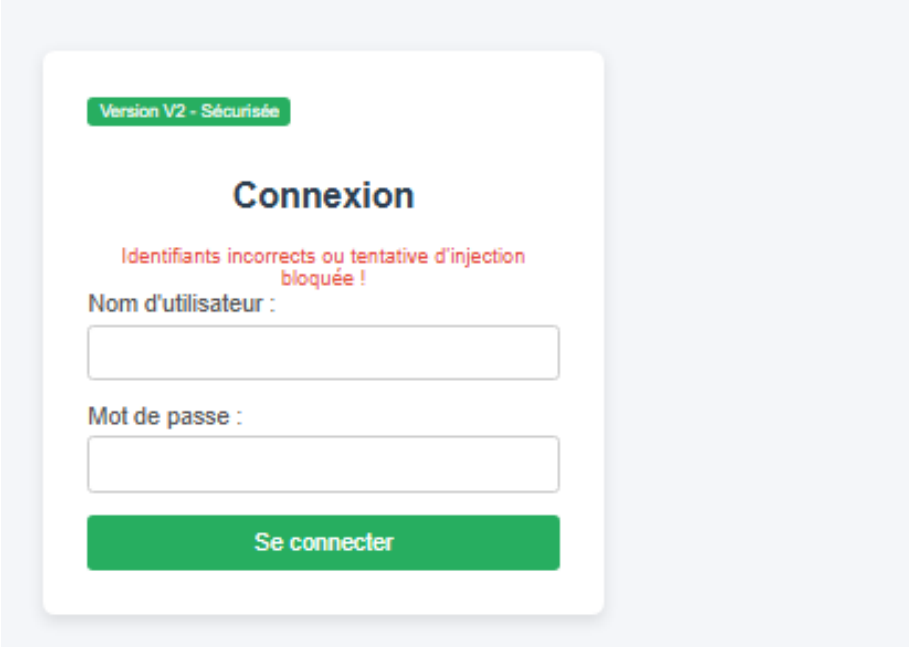


FIGURE 6 – Échec du contournement par injection SQL sur la version sécurisée

4.2 Sécurisation et hachage robuste des mots de passe

L'application intègre désormais la fonction native `password_hash()` à l'enregistrement et `password_verify()` lors de l'authentification.

Listing 4 – Vérification sécurisée avec password_verify()

```
if ($user && password_verify($password, $user['password'])) {
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['username'] = $user['username'];
    header('Location: index_secure.php');
    exit();
} else {
```

```
    $error = "Identifiants incorrects ou tentative bloquee !";
}
```

4.3 Neutralisation des failles XSS

Un échappement systématique des sorties a été mis en place lors de l'affichage dynamique des commentaires.

Listing 5 – Échappement HTML avec htmlspecialchars

```
<p><?= htmlspecialchars($comment['content'], ENT_QUOTES, 'UTF-8');
   ?></p>
```

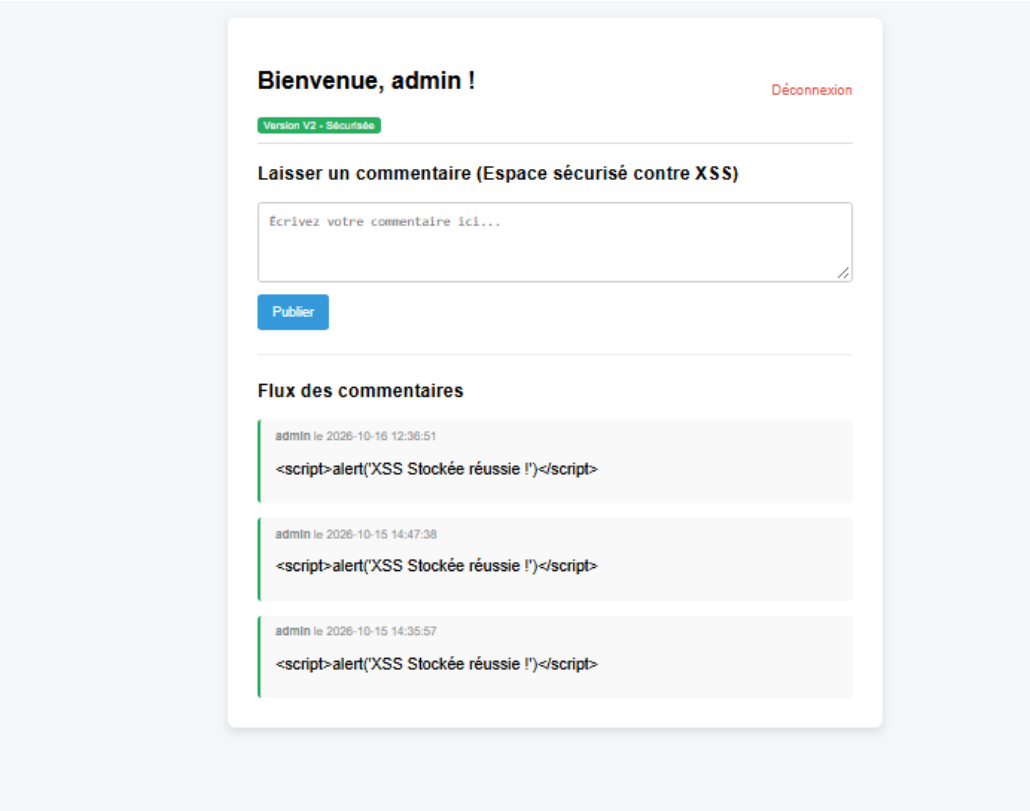


FIGURE 7 – Affichage neutralisé du XSS stocké (converti en texte brut) dans le flux